

INSTITUTO FEDERAL GOIANO - CAMPUS URUTAÍ
SISTEMAS DE INFORMAÇÃO
DISCIPLINA: SISTEMAS DISTRIBUÍDOS
PROFESSOR: JÚNIO CÉSAR DE LIMA

SISTEMAS PUBLICAR-ASSINAR EM SISTEMAS DISTRIBUÍDOS

Comunicação indireta, eventos distribuídos e desacoplamento

Aluno(a): Otávio de Queiroz Franco

Urutaí - GO
2026

SISTEMAS PUBLICAR-ASSINAR EM SISTEMAS DISTRIBUÍDOS

Resumo

Sistemas publicar-assinar, também conhecidos como sistemas *publish/subscribe* ou sistemas baseados em eventos distribuídos, constituem um paradigma importante de comunicação indireta. Nesse modelo, publicadores produzem eventos e assinantes declaram interesse por determinados padrões de eventos, sem que exista uma dependência direta entre quem publica e quem recebe. O serviço de eventos ou broker torna-se responsável por associar publicações e assinaturas, encaminhando notificações aos participantes interessados. Este relatório apresenta os conceitos centrais do modelo publicar-assinar, seu funcionamento, formas de assinatura, questões de implementação e aplicações práticas. Também discute o uso de brokers como RabbitMQ e abstrações como Spring Cloud Stream, destacando a diferença entre grupos de consumidores e o comportamento clássico de pub/sub. Conclui-se que o paradigma é especialmente adequado para arquiteturas orientadas a eventos, disseminação de informação em larga escala e integração de aplicações heterogêneas, embora exija atenção a escalabilidade, ordenação, confiabilidade e observabilidade.

Palavras-chave: Sistemas distribuídos. Publicar-assinar. Pub/sub. Eventos distribuídos. Middleware.

1 Introdução

A comunicação é um dos aspectos fundamentais dos sistemas distribuídos. Como os componentes de uma aplicação podem estar localizados em máquinas diferentes, a coordenação entre eles depende de mecanismos de troca de mensagens capazes de lidar com heterogeneidade, latência, falhas parciais e variações de carga. Entre os diversos paradigmas de comunicação indireta, o modelo publicar-assinar se destaca por permitir que eventos sejam disseminados para múltiplos interessados sem que o publicador conheça diretamente cada assinante.

No modelo publicar-assinar, um publicador envia eventos para um serviço intermediário, enquanto os assinantes registram assinaturas que expressam os tipos de eventos que desejam receber. A tarefa principal do sistema é comparar eventos publicados com assinaturas registradas e entregar notificações aos assinantes compatíveis. Diferentemente de uma fila de mensagens ponto a ponto, na qual uma mensagem normalmente é consumida por apenas um processo, pub/sub é naturalmente um modelo um-para-muitos.

Esse paradigma é amplamente utilizado em sistemas financeiros, monitoramento de infraestrutura, colaboração em tempo real, aplicações móveis, computação ubíqua,

sistemas de notificação, arquiteturas orientadas a eventos e plataformas de streaming. Em todos esses casos, a motivação central é semelhante: separar quem produz a informação de quem a consome, favorecendo flexibilidade, expansão e evolução independente dos componentes.

2 Fundamentos do paradigma publicar-assinar

O modelo publicar-assinar introduz pelo menos três papéis principais: publicadores, assinantes e serviço de eventos. Publicadores são componentes que geram eventos estruturados, como uma atualização de preço, um clique em anúncio, uma alteração de estado ou uma medição de sensor. Assinantes são componentes que expressam interesse por eventos específicos. O serviço de eventos recebe publicações, mantém informações sobre assinaturas e entrega notificações para os assinantes adequados.

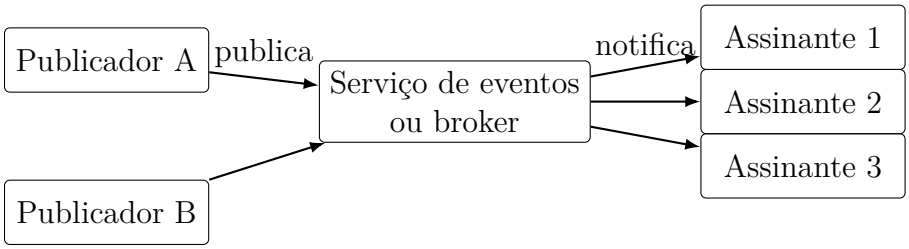


Figura 1: Modelo conceitual de um sistema publicar-assinar.

O desacoplamento é a propriedade central desse modelo. Há desacoplamento espacial porque publicadores não precisam conhecer a identidade dos assinantes. Há também desacoplamento temporal quando o sistema permite que eventos e assinaturas sejam tratados sem exigir presença simultânea de todos os participantes. Além disso, a comunicação costuma ser assíncrona: o publicador envia um evento e segue sua execução, enquanto o middleware se encarrega de entregar notificações.

Essa separação torna pub/sub adequado para ambientes nos quais participantes entram e saem, interesses mudam com frequência e a quantidade de consumidores pode crescer de modo imprevisível. Uma plataforma de monitoramento, por exemplo, pode publicar eventos de CPU, memória, rede e falhas. Diferentes painéis, alarmes e serviços de auditoria podem assinar apenas os eventos que lhes interessam, sem exigir mudanças no código dos produtores.

3 Modelo de programação

O modelo de programação de sistemas publicar-assinar é baseado em um pequeno conjunto de operações. A operação `publish(e)` permite ao publicador disseminar um evento. A operação `subscribe(f)` permite ao assinante declarar interesse por eventos

que correspondam a um filtro. A operação `unsubscribe(f)` remove uma assinatura previamente registrada. A operação `notify(e)` representa a entrega do evento ao assinante.

Alguns sistemas também incluem operações de anúncio. Por meio de `advertise(f)`, um publicador declara previamente a natureza dos eventos que pretende produzir. A operação inversa, `unadvertise(f)`, revoga esse anúncio. Esses mecanismos podem ajudar o middleware a otimizar roteamento, validar contratos e reduzir tráfego desnecessário, especialmente em arquiteturas distribuídas.

O evento, por sua vez, costuma ser uma estrutura com atributos. Ele pode conter tipo, tópico, origem, timestamp, payload e metadados de roteamento. O assinante não precisa receber todos os eventos publicados no sistema; ele recebe apenas aqueles que satisfazem seu filtro. Por isso, a expressividade dos filtros é um ponto decisivo. Sistemas simples podem filtrar apenas por canal; sistemas mais sofisticados podem filtrar por atributos, tipos, padrões contextuais ou eventos compostos.

4 Modelos de assinatura e filtros

A assinatura é o mecanismo pelo qual um assinante expressa interesse. A forma mais simples é o modelo baseado em canal. Nele, publicadores enviam eventos para canais nomeados, e assinantes recebem todos os eventos publicados no canal ao qual se inscreveram. Esse modelo é fácil de implementar, mas oferece baixa expressividade, pois o canal funciona como uma categoria fixa.

O modelo baseado em tópico é semelhante, mas o tópico aparece explicitamente como campo do evento. Um assinante pode declarar interesse por eventos de um tópico como `financeiro.cotacao` ou `sistema.alerta`. Essa estratégia é comum em brokers e plataformas de mensageria porque equilibra simplicidade e organização lógica.

O modelo baseado em conteúdo é mais expressivo. Nesse caso, assinaturas são consultas sobre atributos do evento. Um assinante poderia solicitar eventos cujo tipo seja `alerta`, cuja severidade seja maior que determinado valor e cuja localização pertença a uma região específica. Esse modelo reduz notificações irrelevantes, mas exige mecanismos mais complexos de correspondência e roteamento.

Também há modelos baseados em tipo, nos quais eventos são classificados por tipos ou subtipos ligados à linguagem ou ao modelo de objetos. Em aplicações móveis e ubíquas, podem surgir filtros baseados em contexto, como localização, proximidade, horário ou estado do ambiente. Em cenários mais avançados, o processamento de eventos complexos permite detectar padrões formados por múltiplos eventos relacionados.

Tabela 1: Modelos de assinatura em pub/sub

Modelo	**Ideia principal**	**Característica**
Canal	Eventos são publicados em canais nomeados.	Simples, mas pouco expressivo.
Tópico	Eventos possuem um tópico explícito.	Organiza categorias e facilita roteamento.
Conteúdo	Assinaturas consultam atributos do evento.	Mais flexível, porém mais complexo.
Tipo	Eventos são filtrados por tipos ou subtipos.	Integra-se bem a linguagens e modelos de objetos.
Contexto	Filtros usam localização, tempo ou ambiente.	Útil em computação móvel e ubíqua.

5 Implementação e arquitetura

A implementação de pub/sub pode ser centralizada ou distribuída. Na implementação centralizada, um único broker recebe publicações, mantém assinaturas e encaminha notificações. Essa abordagem é simples e facilita controle administrativo, mas cria um ponto único de falha e pode se tornar gargalo de desempenho quando há muitos publicadores, assinantes ou eventos.

Em uma implementação distribuída, vários brokers cooperam para rotear eventos. Essa rede de intermediários pode replicar informações de assinatura, encaminhar eventos por caminhos otimizados e tolerar melhor falhas de nós específicos. Em arquiteturas ainda mais descentralizadas, os próprios participantes podem atuar como intermediários, formando uma rede peer-to-peer de roteamento de eventos.

O roteamento de eventos é um dos problemas centrais. Uma estratégia simples é a inundação, na qual eventos são encaminhados amplamente e a filtragem ocorre perto do assinante. Essa solução é fácil de entender, mas gera tráfego desnecessário. Uma estratégia mais eficiente é a filtragem na rede de brokers: informações de assinatura são propagadas, e cada broker encaminha apenas eventos que tenham algum caminho para assinantes interessados.

Além do roteamento, sistemas publicar-assinar precisam tratar segurança, concorrência, tolerância a falhas, ordenação, qualidade de serviço e observabilidade. Dependendo do domínio, pode ser aceitável perder eventos ocasionais, como em atualizações frequentes de posição em um jogo. Em outros contextos, como uma sala de negociação, todos os participantes interessados devem receber as mesmas informações e, muitas vezes, na mesma ordem.

6 Exemplo com RabbitMQ e Spring Cloud Stream

Uma forma prática de implementar pub/sub em aplicações Java é usar Spring Cloud Stream com RabbitMQ ou Kafka como broker. O Spring Cloud Stream fornece uma abstração de canais lógicos, permitindo que a aplicação publique e consuma eventos sem lidar diretamente com detalhes de baixo nível do broker.

No exemplo discutido em aula, um *publisher* envia objetos de evento para um destino lógico, como `event-topic`. Um *subscriber* recebe esses eventos por meio de um `Consumer<Evento>`. A configuração no `application.yml` define bindings como `publish-out` e `subscribe-in`, associando canais da aplicação aos destinos do broker. O método `streamBridge.send(...)` publica eventos, enquanto uma função consumidora processa mensagens recebidas.

Um ponto importante é o uso de grupos. Quando vários consumidores usam o mesmo grupo, as mensagens podem ser distribuídas entre eles, criando comportamento semelhante a consumidores concorrentes de uma fila. Isso é útil para escalabilidade, pois apenas uma instância processa cada mensagem dentro do grupo. Quando consumidores usam grupos diferentes, cada grupo recebe sua própria cópia do evento, caracterizando o comportamento clássico de publicar-assinar.

Esse exemplo mostra que pub/sub e filas não são categorias completamente isoladas nas plataformas reais. Um broker pode oferecer ambos os comportamentos dependendo da configuração de destinos, exchanges, tópicos, filas e grupos de consumidores. Por isso, o projeto da arquitetura deve deixar claro se o objetivo é disseminar cada evento para muitos assinantes ou dividir trabalho entre instâncias equivalentes.

7 Vantagens, limitações e cuidados

A primeira vantagem do pub/sub é o desacoplamento. Publicadores podem evoluir sem conhecer todos os assinantes, e novos assinantes podem ser adicionados sem alterar os produtores. Isso favorece arquiteturas extensíveis e facilita a criação de novos serviços consumidores, como painéis, notificadores, auditores e módulos de análise.

Outra vantagem é a disseminação eficiente de informação. Quando muitos componentes precisam reagir ao mesmo evento, o paradigma publicar-assinar evita que o publicador implemente chamadas diretas para todos os interessados. O broker centraliza ou coordena a entrega, tornando o sistema mais flexível e alinhado a arquiteturas orientadas a eventos.

Entretanto, pub/sub também introduz desafios. A visibilidade do fluxo pode diminuir, pois publicador e assinante não estão diretamente conectados. Erros de roteamento, filtros incorretos, eventos não processados e consumidores lentos podem ser mais difíceis de diagnosticar sem métricas, logs e rastreamento distribuído. Além disso, garantias de

entrega, ordenação e persistência dependem fortemente da configuração do broker e dos requisitos da aplicação.

Também é necessário projetar contratos de eventos com cuidado. O formato dos eventos precisa ser estável o suficiente para permitir evolução sem quebrar assinantes existentes. Mudanças de schema, campos opcionais, versionamento e compatibilidade tornam-se preocupações importantes. Assim, pub/sub simplifica a comunicação direta entre componentes, mas exige disciplina na governança dos eventos.

8 Conclusão

Sistemas publicar-assinar são uma forma poderosa de comunicação indireta em sistemas distribuídos. Ao separar publicadores de assinantes por meio de um serviço de eventos, esse paradigma reduz acoplamento, favorece disseminação de informação e permite arquiteturas mais flexíveis.

O estudo do modelo mostra que sua força está na combinação entre eventos, assinaturas e filtros. Modelos simples, como canal e tópico, são fáceis de implementar e suficientes para muitos sistemas. Modelos baseados em conteúdo, tipo e contexto oferecem maior expressividade, mas exigem infraestrutura mais sofisticada de correspondência e roteamento.

Na prática, tecnologias como RabbitMQ, Kafka e Spring Cloud Stream mostram que pub/sub é essencial para sistemas modernos baseados em eventos. Ainda assim, seu uso deve considerar escalabilidade, confiabilidade, observabilidade, ordenação e evolução dos contratos de eventos. Quando bem projetado, o paradigma publicar-assinar permite que sistemas distribuídos cresçam de forma mais modular, reativa e adaptável.